

Notas de Estudio - Período 3 Machine Learning Básico con Scikit-Learn

1 Conceptos Fundamentales

1.1 ¿Qué es un Modelo?

En este curso, un **modelo** se compara con un **entrenador que aprende patrones**. No es simplemente un observador pasivo, sino un agente que:

- Analiza datos históricos
- Identifica patrones y relaciones
- Genera predicciones basadas en lo aprendido

1.2 Entrenar un Modelo

Entrenar un modelo significa **ajustar parámetros internos usando datos históricos**.

NO es:

- Memorizar únicamente resultados exactos
- Editar manualmente las respuestas
- Mezclar datos sin orden

El entrenamiento es un proceso iterativo donde el modelo aprende de ejemplos pasados para generalizar a nuevos casos.

2 División de Datos (Train/Test Split)

2.1 Función Principal

En scikit-learn, la función para dividir datos es: `train_test_split()`

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
```

2.2 Ejemplo Práctico

Si divides datos con `test_size=0.2` y tienes 250 partidos:

- **Entrenamiento:** 200 partidos (80%)
- **Prueba:** 50 partidos (20%)

3 Overfitting (Sobreajuste)

3.1 Analogía Futbolística

Overfitting es como un entrenador que memoriza jugadas sin entender el juego.

3.2 Síntomas de Overfitting

Un síntoma clásico:

- Alta precisión en entrenamiento
- Baja precisión en prueba

Esto indica que el modelo memorizó los datos de entrenamiento pero no generalizó bien.

4 Modelos de Clasificación

4.1 Modelos Principales

1. Regresión Logística (LogisticRegression)

- Para clasificación binaria básica
- Más simple e interpretable
- Buen punto de partida

2. Random Forest

- Ensemble de árboles de decisión
- Generalmente más robusto
- Puede capturar relaciones no lineales

```
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
```

```
# Regresión Logística
lr = LogisticRegression()
lr.fit(X_train, y_train)
```

```
# Random Forest
rf = RandomForestClassifier()
rf.fit(X_train, y_train)
```

5 Características (Features)

5.1 Selección de Características

Si tu DataFrame tiene columnas `goles_local`, `goles_visitante`, `tiros_local`, una lista válida es:

```
features = ['goles_local', 'goles_visitante', 'tiros_local']
```

IMPORTANTE: Usar comillas y nombres exactos de columnas.

5.2 ¿Por Qué No Usar Demasiadas Columnas Irrelevantes?

Puede introducir ruido y empeorar la generalización.

Variables irrelevantes:

- Añaden complejidad innecesaria
- Pueden confundir al modelo
- Reducen la interpretabilidad

5.3 Importancia de Variables en Random Forest

La **importancia de variables** indica la **contribución a reducir errores de predicción**.

Variables con mayor importancia:

- Aportan más a las divisiones internas de los árboles
- Son más útiles para las predicciones

```
# Obtener importancias
importances = rf.feature_importances_
```

6 Variables Derivadas

6.1 ¿Qué Son?

Crear `total_goles = goles_local + goles_visitante` es un ejemplo de **variable derivada simple**.

6.2 Ejemplos Útiles

1. Total de Goles

```
df['total_goles'] = df['goles_local'] + df['goles_visitante']
```

2. Diferencia de Goles

```
df['diferencia_goles'] = df['goles_local'] - df['goles_visitante']
```

6.3 Interpretación

Si `diferencia_goles` mejora la precisión:

- La diferencia de goles es **predictiva para el resultado**
- Captura información relevante sobre el desempeño relativo

Si eliminar una variable mantiene la precisión igual:

- La variable **no aportaba** información útil

7 Métricas de Evaluación

7.1 Precisión (Accuracy)

Precisión en clasificación:

$$\text{Precisión} = \frac{\text{Predicciones correctas}}{\text{Total predicciones}}$$

Ejemplo: Si tu modelo acierta 34 de 50 partidos:

$$\text{Precisión} = \frac{34}{50} = 0.68 = 68\%$$

```
from sklearn.metrics import accuracy_score
accuracy = accuracy_score(y_test, y_pred)
```

7.2 Matriz de Confusión

Una **matriz de confusión** muestra:

- Verdaderos positivos (VP)
- Falsos positivos (FP)
- Verdaderos negativos (VN)
- Falsos negativos (FN)

```
from sklearn.metrics import confusion_matrix
cm = confusion_matrix(y_test, y_pred)
```

	Predicho: Positivo	Predicho: Negativo
Real: Positivo	VP	FN
Real: Negativo	FP	VN

8 Baseline y Comparación

8.1 ¿Qué es el Baseline?

El **baseline** es predecir siempre la clase mayoritaria.

¿Por qué comparar con baseline?

- Para saber si el modelo supera predecir siempre la clase mayoritaria
- Establece un punto de referencia mínimo

8.2 Ejemplo de Baseline

Si tienes 18 victorias locales de 40 partidos:

$$\text{Baseline} = \frac{18}{40} = 0.45 = 45\%$$

8.3 Interpretación de Resultados

Si tu baseline es 0.60 y tu modelo 0.62:

- **Mejora marginal** que debe analizarse si vale el esfuerzo
- No es una victoria aplastante
- Considera si la complejidad adicional justifica el beneficio

9 Comparación de Modelos

9.1 Ejemplo Práctico

Entrenas Regresión Logística y Random Forest:

- RF obtiene 0.70
- RL obtiene 0.68

Acción razonable inicial: Conservar ambos y explicar diferencia ligera

9.2 Simplicidad vs. Complejidad

Si un modelo más sencillo (RL) rinde casi igual que RF:

- **Puede preferirse** el modelo simple
- Suele ser más interpretable
- Es suficiente para el problema

Razón: La interpretabilidad y simplicidad son valiosas cuando el rendimiento es similar.

9.3 ¿Por Qué Limitar el Número de Modelos?

En este curso se limitan los modelos para:

- **Evitar confundir con exceso de complejidad**
- **Centrarse en fundamentos**
- Desarrollar intuición antes de explorar técnicas avanzadas

10 Análisis de Cambios en Variables

10.1 Incremento de Precisión

Si añadir `diferencia_goles` sube la precisión de 0.62 a 0.66:

- **Incremento absoluto:** $0.66 - 0.62 = 0.04$
- **Incremento porcentual:** 4 puntos porcentuales

10.2 Lecciones Aprendidas

Si la precisión mejora al añadir una variable:

- La variable contiene información predictiva útil

Si la precisión cae al añadir una variable:

- Variables irrelevantes pueden introducir ruido
- Es mejor mantener el modelo más simple

Si la precisión se mantiene igual:

- La variable no aportaba información nueva
- Puede eliminarse para simplificar

11 Resumen de Flujo de Trabajo

1. Preparar datos

- Cargar dataset
- Crear variables derivadas si es necesario

2. Dividir datos

- Usar `train_test_split()`
- Típicamente 80/20 o 70/30

3. Entrenar modelos

- Probar modelos simples primero (LogisticRegression)
- Comparar con modelos más complejos (RandomForest)

4. Evaluar

- Calcular precisión
- Revisar matriz de confusión
- Comparar con baseline

5. Analizar importancia de variables

- Identificar características más útiles
- Eliminar ruido si es necesario

6. Iterar

- Probar nuevas variables derivadas
- Ajustar si vale la pena
- Mantener simplicidad cuando sea posible